

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1. Introduction:

This project aims to replace manual reporting to ensure 100% compliance with Malaysia's 2026 Carbon Tax regulations while reducing their carbon emissions. However, many organizations struggle to accurately calculate their carbon emissions because relevant data is scattered across different operational areas such as transportation fuel usage, electricity consumption, and waste management. Traditional carbon accounting methods often require manual data collection and complex calculations, which are time-consuming, error-prone, and difficult to scale.

1.1. Purpose:

The purpose of this System Analysis Documentation (SAD) is to provide a detailed technical description of the AI-based Organizational Carbon Tax Monitoring System. This system is designed to assist organizations in monitoring environmental impact, calculating carbon tax liabilities, and generating compliance reports using artificial intelligence technologies. The documentation outlines the system architecture, operational logic, data flows, and technical components required to implement the platform.

The system integrates several modules including a data extraction engine, a knowledge base database, an AI reasoning service, and a web-based interface built using Streamlit. These modules work together to collect environmental data such as fuel consumption, electricity usage, and waste generation, analyze this information using AI models, and generate reports that support management decisions and regulatory compliance.

Another purpose of this documentation is to provide stakeholders, developers, and system analysts with a clear understanding of the system's operational workflow. This includes the interactions between the user interface, database, and AI models that perform advanced reasoning tasks based on organizational procurement discussions and environmental regulations.

Ultimately, the system aims to support organizations in transitioning toward more sustainable operations while ensuring compliance with emerging carbon taxation policies.

1.2. Background:

Carbon taxation policies are increasingly being implemented worldwide as governments attempt to reduce greenhouse gas emissions and encourage sustainable industrial practices. Organizations must now monitor their environmental footprint more carefully, particularly in areas such as fuel consumption, electricity usage, and industrial waste production.

However, many companies face challenges in collecting and analyzing environmental data efficiently. Data related to emissions is often scattered across different departments and systems, making it difficult

to produce accurate environmental reports. Additionally, decision-making regarding environmentally friendly procurement options often requires complex lifecycle analysis and cost comparisons.

The proposed system addresses these challenges by combining automated data extraction, centralized environmental databases, and AI-driven analysis. The system allows users to upload policy documents, extract relevant environmental information, and analyze operational decisions using artificial intelligence models.

For example, procurement discussions between company staff can be analyzed by the AI system to evaluate carbon emissions and calculate tax implications. This allows decision makers to compare environmentally friendly options with traditional alternatives while considering both financial costs and carbon tax liabilities.

By integrating environmental intelligence directly into operational workflows, the system supports organizations in making data-driven sustainability decisions.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Company Executives (CEO / Directors)	Strategic decision makers responsible for procurement and sustainability strategy	Expect high-level reports summarizing carbon emissions, tax impact, and ESG implications
Procurement Managers	Evaluate suppliers and materials used in projects	Expect accurate lifecycle emission calculations and cost comparisons
Operations Staff	Input operational data such as logistics information or procurement details	Expect an easy-to-use system for uploading data and interacting with AI analysis
System Developers	Design and maintain the system infrastructure	Expect clear architecture, documentation, and maintainable code structure
Sustainability Analysts	Evaluate environmental performance and ESG metrics	Expect access to emission data, lifecycle analysis, and environmental reports

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

The system architecture follows a modular design consisting of three major layers: the presentation layer, the application logic layer, and the data storage layer. The presentation layer provides a web interface built using Streamlit that allows users to interact with the system. Through this interface, users can log in, communicate via a chat interface, upload policy documents, and generate environmental analysis reports.

The application logic layer handles data processing, artificial intelligence analysis, and system operations. This layer includes the AI reasoning engine, document processing modules, and database interaction logic. The AI engine analyzes procurement discussions and environmental datasets to produce compliance reports and strategic recommendations.

The data layer stores structured environmental data and extracted information from policy documents. The database is implemented using SQLite, which stores knowledge base information, material lifecycle analysis data, and operational environmental metrics. By separating the system into layers, the architecture ensures scalability, maintainability, and easier integration with external services.

2.1.2 LLM as Service Layer

The system integrates a large language model (LLM) as an external service responsible for performing advanced reasoning and report generation. The LLM analyzes chat history, procurement discussions, and environmental data stored in the database to produce structured reports for decision makers.

The AI service is accessed through an API endpoint provided by the Ilmu-GLM platform. When a user requests analysis, the system constructs a prompt containing relevant organizational data, lifecycle assessment metrics, and procurement details. This prompt is then sent to the AI model, which generates strategic recommendations and compliance reports.

Using an LLM as a service layer allows the system to perform complex reasoning tasks without requiring heavy computational resources on the local server. The AI model can interpret human conversations, analyze environmental data, and generate meaningful insights for executives and compliance officers.

2.1.3 Dependency Diagram

(a) How are the prompts being constructed and sent to the GLM API?

```

def run_ai_analysis(user_message: str, conversation_history: list) -> str:
    facts = get_relevant_facts(user_message)
    kb_data = search_knowledge_base(user_message)
    system = build_system_prompt(facts, kb_data)
    messages = [{"role": "system", "content": system}]
    for msg in conversation_history[-10:]:
        role = "user" if msg["sender"] != "AI" else "assistant"
        messages.append({"role": role, "content": msg["text"]})
    messages.append({"role": "user", "content": user_message})
    try:
        response = requests.post(
            f"{CUSTOM_BASE_URL}/chat/completions",
            headers={"Authorization": f"Bearer {Z_AI_API_KEY}", "Content-Type": "application/json"},
            json={"model": MODEL_NAME, "messages": messages, "temperature": 0.3, "max_tokens": 800},
            timeout=30)
        response.raise_for_status()
        return response.json()["choices"][0]["message"]["content"]
    except requests.exceptions.HTTPError:
        return f"⚠ HTTP Error {response.status_code}: {response.text}"
    except Exception as e:
        return f"⚠ AI Error: {str(e)}"

def build_system_prompt(facts, kb_data):
    return f"""You are GreenLoom AI – a sustainability and trade-compliance analyst for a Malaysian manufacturing firm.

STRUCTURED DATA:
- Materials: {facts['materials']} or 'No records.'
- LCA Metrics: {facts['lca']} or 'No records.'

POLICY EXCERPTS:
{kb_data}

RULES:
1. Base answers strictly on provided data.
2. If insufficient: say "Insufficient data – please upload the relevant policy PDF."
3. CEO queries: strategic insight (cost, carbon savings, competitive edge).
4. Compliance queries: cite source and page number.
5. Be concise and professional. Use bullet points for reports."""

```

(b) What goes into the context window?

```

def build_system_prompt(facts, kb_data):
    return f"""You are GreenLoom AI – a sustainability and trade-compliance analyst for a Malaysian manufacturing firm.

STRUCTURED DATA:
- Materials: {facts['materials']} or 'No records.'
- LCA Metrics: {facts['lca']} or 'No records.'

POLICY EXCERPTS:
{kb_data}

RULES:
1. Base answers strictly on provided data.
2. If insufficient: say "Insufficient data – please upload the relevant policy PDF."
3. CEO queries: strategic insight (cost, carbon savings, competitive edge).
4. Compliance queries: cite source and page number.
5. Be concise and professional. Use bullet points for reports."""

```

(c) How is your system receiving, parsing the responses and passing to the next system component?

```
try:
    response = requests.post(
        f"{CUSTOM_BASE_URL}/chat/completions",
        headers={"Authorization": f"Bearer {Z_AI_API_KEY}", "Content-Type": "application/json"},
        json={"model": MODEL_NAME, "messages": messages, "temperature": 0.3, "max_tokens": 800},
        timeout=30)
    response.raise_for_status()
    return response.json()["choices"][0]["message"]["content"]
except requests.exceptions.HTTPError:
    return f"⚠ HTTP Error {response.status_code}: {response.text}"
except Exception as e:
    return f"⚠ AI Error: {str(e)}"
```

(d) Where is the limitation of tokens enforced or input is chunked before it actually reaches GLM?

```
try:
    response = requests.post(
        f"{CUSTOM_BASE_URL}/chat/completions",
        headers={"Authorization": f"Bearer {Z_AI_API_KEY}", "Content-Type": "application/json"},
        json={"model": MODEL_NAME, "messages": messages, "temperature": 0.3, "max_tokens": 800},
        timeout=30)
```

(e) Outline all the major API calls between the system components.

```
Z_AI_API_KEY = "..."
CUSTOM_BASE_URL = "https://api.ilmu.ai/v1"
MODEL_NAME = "ilmu-glm-5.1"

# — Groq Vision API (for Wastage Identifv) —
GROQ_API_KEY = "..."
GROQ_BASE_URL = "https://api.groq.com/openai/v1"
GROQ_VISION_MODEL = "meta-llama/llama-4-scout-17b-16e-instruct"
```

(f) Show how frontend, backend, services, database and external API interact with each other.

UI → Backend → DB → AI → UI

Frontend:


```

with st.expander("🔴 SCOPE 1 – Direct Emissions (Company Vehicles & Fuel)", expanded=True):
    col1, col2 = st.columns(2)
    with col1:
        petrol_imgs = st.file_uploader("📁 Petrol Receipts (JPG/PNG – multiple allowed)", type=["jpg", "jpeg", "png"])
        if petrol_imgs:
            for pi in petrol_imgs:
                st.image(pi, caption=pi.name, use_container_width=True)
            if st.button("Extract Petrol Receipt(s)", key="ext_petrol"):
                if petrol_imgs:
                    total_petrol, total_diesel = 0.0, 0.0
                    for pi in petrol_imgs:
                        with st.spinner(f"Reading {pi.name}..."):
                            ext = extract_receipt_with_vision(pi, "petrol fuel")
                            if "error" not in ext:
                                litres = float(ext.get("litres") or 0)
                                fuel_type = str(ext.get("fuel_type", "petrol")).lower()
                                if "diesel" in fuel_type:
                                    total_diesel += litres
                                else:
                                    total_petrol += litres
                            else:
                                st.warning(f"{pi.name}: {ext['error']}")
                    if total_petrol > 0:
                        st.session_state.s1_petrol_l += total_petrol
                        st.session_state["_toast_petrol"] = True
                    if total_diesel > 0:
                        st.session_state.s1_diesel_l += total_diesel
                        st.session_state["_toast_diesel"] = True
                    st.session_state["calc_form_key"] = fk + 1
                    st.rerun()
                else:
                    st.warning("Upload at least one receipt image first.")
    with col2:
        diesel_imgs = st.file_uploader("📁 Diesel Receipts (JPG/PNG – multiple allowed)", type=["jpg", "jpeg", "png"])
        if diesel_imgs:
            for di in diesel_imgs:
                st.image(di, caption=di.name, use_container_width=True)
            if st.button("Extract Diesel Receipt(s)", key="ext_diesel"):
                if diesel_imgs:
                    total_litres = 0.0
                    for di in diesel_imgs:
                        with st.spinner(f"Reading {di.name}..."):
                            ext = extract_receipt_with_vision(di, "diesel fuel")
                            if "error" not in ext:
                                total_litres += float(ext.get("litres") or 0)
                            else:
                                st.warning(f"{di.name}: {ext['error']}")
                    st.session_state.s1_diesel_l += total_litres
                    st.session_state["_toast_diesel"] = True
                    st.session_state["calc_form_key"] = fk + 1
                    st.rerun()
                else:
                    st.warning("Upload at least one receipt image first.")

```

```

c1, c2, c3 = st.columns(3)
with c1:
    petrol_l = st.number_input("Petrol (litres)", min_value=0.0,
                               value=st.session_state.s1_petrol_l, step=1.0, key=f"petrol_l_in_{fk}")
with c2:
    diesel_l = st.number_input("Diesel (litres)", min_value=0.0,
                               value=st.session_state.s1_diesel_l, step=1.0, key=f"diesel_l_in_{fk}")
with c3:
    lpg_kg = st.number_input("LPG (kg)", min_value=0.0,
                             value=st.session_state.s1_lpg_kg, step=1.0, key=f"lpg_kg_{fk}")

s1_kg = petrol_l*EF["petrol_litre"] + diesel_l*EF["diesel_litre"] + lpg_kg*EF["lpg_kg"]
st.info(f"**Scope 1 Subtotal: {s1_kg/1000:.4f} t CO2e**")

```

Backend:

```

def calculate_carbon_tax(scope1, scope2, scope3_logistics, scope3_waste, scope3_materials, offset_t):
    scope3 = scope3_logistics + scope3_waste + scope3_materials
    total_kg = scope1 + scope2 + scope3
    total_t = total_kg / 1000.0
    net_t = max(0, total_t - offset_t)
    taxable_t = max(0, net_t - FREE_ALLOWANCE_TONNES)
    tax_rm = taxable_t * CARBON_TAX_RATE_RM
    offset_cost = offset_t * OFFSET_PRICE_RM
    return {
        "scope1_t": scope1 / 1000,
        "scope2_t": scope2 / 1000,
        "scope3_t": scope3 / 1000,
        "scope3_logistics_t": scope3_logistics / 1000,
        "scope3_waste_t": scope3_waste / 1000,
        "scope3_materials_t": scope3_materials / 1000,
        "total_t": total_t,
        "offset_t": offset_t,
        "net_t": net_t,
        "allowance_t": FREE_ALLOWANCE_TONNES,
        "taxable_t": taxable_t,
        "tax_rate": CARBON_TAX_RATE_RM,
        "tax_rm": tax_rm,
        "offset_cost_rm": offset_cost,
        "net_tax_rm": max(0, tax_rm - offset_cost),
    }

```



```
def process_uploaded_file(uploaded_file):
    """Redo extraction for a new policy PDF and update the knowledge base."""
    conn = sqlite3.connect(DB_NAME)
    c = conn.cursor()
    # Table schema from your Data_extract.py
    c.execute('CREATE TABLE IF NOT EXISTS knowledge_base
              (source TEXT, page_num INTEGER, content TEXT)')

    with pdfplumber.open(uploaded_file) as pdf:
        for i, page in enumerate(pdf.pages):
            # Extract Tables (for HS Codes, Prices) [cite: 3, 238, 627]
            tables = page.extract_tables()
            for table in tables:
                for row in table:
                    clean_row = [str(cell).strip() for cell in row if cell]
                    if clean_row:
                        c.execute("INSERT INTO knowledge_base VALUES (?, ?, ?)",
                                (uploaded_file.name, i + 1, " | ".join(clean_row)))
            # Extract Raw Text (for Law/Policy acts) [cite: 23, 99, 736]
            text = page.extract_text()
            if text:
                c.execute("INSERT INTO knowledge_base VALUES (?, ?, ?)",
                        (uploaded_file.name, i + 1, text.strip()))
    conn.commit()
    conn.close()
    return f"✅ Database updated with {uploaded_file.name}."
```

```
def save_material_to_db(item: str, qty_kg: float, carbon_factor: float, hs_code: str = ""):
    """Save or update a material purchase in the materials table.
    NOTE: 'price' column stores qty_kg - legacy schema kept for compatibility."""
    conn = sqlite3.connect(DB_NAME)
    c = conn.cursor()
    # 'price' column stores qty_kg - must use correct column name in SELECT
    c.execute("SELECT price FROM materials WHERE LOWER(item) = LOWER(?)", (item,))
    existing = c.fetchone()
    if existing:
        new_qty = (existing[0] or 0) + qty_kg
        c.execute("UPDATE materials SET price = ?, carbon_factor = ?, hs_code = ? WHERE LOWER(item) = LOWER(?)",
                (new_qty, carbon_factor, hs_code, item))
    else:
        c.execute("INSERT INTO materials (item, price, carbon_factor, hs_code) VALUES (?, ?, ?, ?)",
                (item, qty_kg, carbon_factor, hs_code))
    conn.commit()
    conn.close()
```

(g) Highlight the distributed components if the system is planning to be deployed across multiple services.

```
def get_supplier_ai_recommendation(supplier_log: list) -> str:
    """Ask AI to recommend the best supplier based on carbon + cost data."""
    if not supplier_log:
        return "No supplier data available yet."

    summary = "\n".join([
        f"- {r[0]}: {r[1]} trips, {r[2]:.0f} km total, {r[3]:.1f}L diesel, "
        f"{r[4]:.2f} kg CO2e, RM {r[5]:.2f} logistics cost"
        for r in supplier_log
    ])

    prompt = f"""You are a sustainability procurement analyst for a Malaysian manufacturer.

SUPPLIER LOGISTICS CARBON DATA:
{summary}

Carbon tax rate: RM 15 per tonne CO2e.

Provide a SHORT recommendation (max 150 words):
1. Which supplier has the LOWEST carbon footprint per trip?
2. Which supplier is most cost-effective when carbon tax is included?
3. One actionable suggestion to reduce logistics emissions."""
```

```
def calculate_carbon_tax(scope1, scope2, scope3_logistics, scope3_waste, scope3_materials, offset_t):
    scope3 = scope3_logistics + scope3_waste + scope3_materials
    total_kg = scope1 + scope2 + scope3
    total_t = total_kg / 1000.0
    net_t = max(0, total_t - offset_t)
    taxable_t = max(0, net_t - FREE_ALLOWANCE_TONNES)
    tax_rm = taxable_t * CARBON_TAX_RATE_RM
    offset_cost = offset_t * OFFSET_PRICE_RM
    return {
        "scope1_t": scope1 / 1000,
        "scope2_t": scope2 / 1000,
        "scope3_t": scope3 / 1000,
        "scope3_logistics_t": scope3_logistics / 1000,
        "scope3_waste_t": scope3_waste / 1000,
        "scope3_materials_t": scope3_materials / 1000,
        "total_t": total_t,
        "offset_t": offset_t,
        "net_t": net_t,
        "allowance_t": FREE_ALLOWANCE_TONNES,
        "taxable_t": taxable_t,
        "tax_rate": CARBON_TAX_RATE_RM,
        "tax_rm": tax_rm,
        "offset_cost_rm": offset_cost,
        "net_tax_rm": max(0, tax_rm - offset_cost),
    }
```

2.1.4 Sequence Diagram

(a) Walk through one user interaction flow or customer journey with the stakeholders

```
def render_carbon_calculator():
    st.title("🌿 Carbon Tax Calculator")
    st.caption("Malaysian Carbon Capture, Utilisation and Storage Act 2025 – RM 15/tonne CO2e")

    if st.button("← Back to Dashboard"):
        st.session_state.show_carbon = False
        st.session_state.show_dashboard = True
        st.rerun()

    st.divider()

    # — Session state for extracted values —
    for k, v in [("s1_petrol_l", 0.0), ("s1_diesel_l", 0.0), ("s1_lpg_kg", 0.0),
                ("s2_kwh", 0.0), ("s3_km", 0.0), ("s3_log_fuel", 0.0)]:
        if k not in st.session_state:
            st.session_state[k] = v

    # Form key – increments on Clear to force widget re-render
    fk = st.session_state.get("calc_form_key", 0)

    # Show extraction success toasts after rerun
    if st.session_state.pop("_toast_petrol", None):
        st.toast(f"✅ {st.session_state.s1_petrol_l}L petrol extracted", icon="🚗")
    if st.session_state.pop("_toast_diesel", None):
        st.toast(f"✅ {st.session_state.s1_diesel_l}L diesel extracted", icon="🚚")
    if st.session_state.pop("_toast_elec", None):
        st.toast(f"✅ {st.session_state.s2_kwh} kWh extracted", icon="⚡")
    if st.session_state.pop("_toast_logistics", None):
        st.toast(f"✅ {st.session_state.s3_km}km / {st.session_state.s3_log_fuel}L extracted", icon="🚚")
```

(b) Show the interactions step by step

```
with st.expander("🔴 SCOPE 1 – Direct Emissions (Company Vehicles & Fuel)", expanded=True):
    col1, col2 = st.columns(2)
    with col1:
        petrol_imgs = st.file_uploader("📁 Petrol Receipts (JPG/PNG – multiple allowed)", type=["jpg", "jpeg", "png"])
        if petrol_imgs:
            for pi in petrol_imgs:
                st.image(pi, caption=pi.name, use_container_width=True)

            if st.button("Extract Petrol Receipt(s)", key="ext_petrol"):
                if petrol_imgs:
                    total_petrol, total_diesel = 0.0, 0.0
                    for pi in petrol_imgs:
                        with st.spinner(f"Reading {pi.name}..."):
                            # ... (code for reading receipts) ...
```



```
with st.spinner(f"Reading {pi.name}..."):
    ext = extract_receipt_with_vision(pi, "petrol fuel")
    if "error" not in ext:
        litres = float(ext.get("litres") or 0)
        fuel_type = str(ext.get("fuel_type", "petrol")).lower()
        if "diesel" in fuel_type:
            total_diesel += litres
        else:
            total_petrol += litres
```

```
resp = requests.post(
    f"{GROQ_BASE_URL}/chat/completions",
    headers={"Authorization": f"Bearer {GROQ_API_KEY}", "Content-Type": "application/json"},
    json={
        "model": GROQ_VISION_MODEL,
        "messages": [{
            "role": "user",
            "content": [
                {"type": "text", "text": prompt},
                {"type": "image_url", "image_url": {"url": f"data:{mime};base64,{b64_img}"}}
            ]
        }],
        "temperature": 0.1,
        "max_tokens": 200
    },
    timeout=30
)
```

```
resp.raise_for_status()
raw = resp.json()["choices"][0]["message"]["content"].strip()
raw = re.sub(r"```(?:json)?\s*", "", raw).replace("```", "").strip()
match = re.search(r"\[.*\]", raw, re.DOTALL)
if match:
    raw = match.group(0)
    parsed = json.loads(raw)
    if isinstance(parsed, list):
        return parsed
except Exception:
    pass
return []
```

```
if total_petrol > 0:
    st.session_state.s1_petrol_l += total_petrol
    st.session_state["_toast_petrol"] = True
if total_diesel > 0:
    st.session_state.s1_diesel_l += total_diesel
    st.session_state["_toast_diesel"] = True
st.session_state["calc_form_key"] = fk + 1
st.rerun()
else:
    st.warning("Upload at least one receipt image first.")
```

(c) Demonstrate each key system features operating from start to finish

Log In



GreenLoom Corporate Portal

Sustainability & Compliance AI Edition

Select Profile

Yip

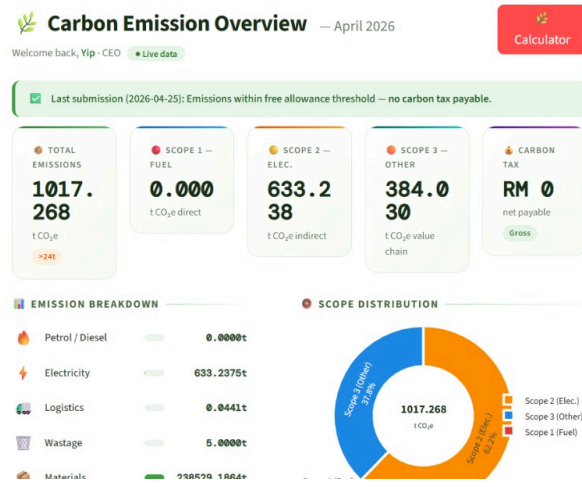
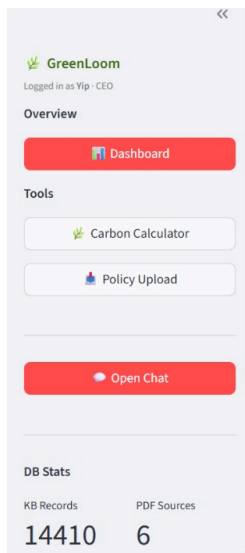
▼

Enter PIN

👁

Login

Front page:



Group chat:



Yip: Chow mentioned we might be able to register steel under BCX carbon credits through MGTC. That could offset the higher production emissions. Look into that too.



Chow: Yep, it's worth a look before we lock in. @AI, pull the full picture on both PPVC options. Check the LCA database, calculate the carbon tax under CCUS Act 2025, and give us a recommendation Boss can sign off on.



Tan: One more thing, for Sepang, is there any lead time buffer we can build in without hurting the overall programme? @AI check if Option B timing is actually a blocker given we can shift the pour.



Yip: @AI, read everything above and give me a final call on both Sepang and Johor. I want the carbon numbers from our database, the tax calc, and your recommendation. Make it something I can bring to the board.

Type your message... (use @AI to get an AI reply)



Information extraction

[← Back to Dashboard](#)

GreenLoom AI — Procurement Recommendation Report

Date: 25 April 2026 | Prepared for: Yip (CEO) | Projects: Sepang Steel Order & Johor PPVC Decision

1. Procurement Items Discussed

Item	Quantity	Price (RM)	HS Code	Status
Structural Steel Beams (Sepang)	15,000,000 kg	See options below	7216.32.000	Decision pending
PPVC Modules — Concrete (Johor)	1,330,000 kg	2,050,000	—	Decision pending
				Decision

Platform for user image upload

▼  SCOPE 1 — Direct Emissions (Company Vehicles & Fuel)

 Petrol Receipts (JPG/PNG — multiple allowed)

 Upload

200MB per file • JPG, JPEG, PNG, WEBP

Extract Petrol Receipt(s)

Petrol (litres)

0.00

–

+

 Diesel Receipts (JPG/PNG — multiple allowed)

 Upload

200MB per file • JPG, JPEG, PNG, WEBP

Extract Diesel Receipt(s)

Diesel (litres)

0.00

–

+

LPG (kg)

0.00

–

+

Scope 1 Subtotal: 0.0000 t CO₂e

▼  SCOPE 2 — Indirect Emissions (Electricity)

 Electricity Bills (JPG/PNG — multiple allowed)

Report generation:

 Save Submission Clear All Inputs

✓ Saved to database.  Report:
carbon_tax_report_20260425_231206.pdf

 Submission Reports carbon_tax_report_20260425_231206.pdf Download

2.2. Technological Stack

2.2.1. Frontend - Streamlit

© UMHackathon 2026

Email: umhackathon@um.edu.my

- 2.2.2. Backend - Python
- 2.2.3. Database - SQLite
- 2.2.4. Cloud/Deployment - Streamlit Community Cloud

2.3. Key Data Flows

- 2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)

2.3.1.2. Normalized Database Schema

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

#	Features	Description
1	Petrol Receipt Upload	Logistics or transport personnel can upload petrol receipts to record fuel purchases. The backend system extracts important information such as fuel quantity and price using data extraction tools.
2	Electricity Consumption Integration	The electricity application collects electricity consumption data and feeds the information directly into the AI system. This allows the system to calculate carbon emissions based on energy usage.
3	Waste Image Scanning	Users can capture images of waste materials through the application. AI-based image analysis estimates the weight and type of waste generated.
4	Carbon Tax Calculation	Based on calculated carbon emissions and applicable tax rates, the system estimates the organization's carbon tax liability.
5	Carbon Tax Report Generation	The system generates structured carbon tax reports summarizing emission sources, total emissions, and estimated tax payments.

3.2. Non-Functional Requirements (NFRs)

Quality	Requirement	Implementation
Performance	The system should process environmental data, AI analysis, and carbon tax report generation within a reasonable response time to ensure a smooth user experience.	Backend processing is optimized using Python, while database queries are kept lightweight using SQLite. AI analysis is

		handled through API calls to reduce local processing load.
Security	The system must protect sensitive organizational data such as petrol usage, electricity consumption, and waste records from unauthorized access.	User authentication and role-based access control ensure that only authorized personnel can access specific system features and environmental reports.
Reliability	The system should consistently produce accurate carbon emission calculations and maintain stable performance during regular operations.	Error handling mechanisms and database transaction management are implemented to maintain data integrity and prevent system failures.
Usability	The application interface should be easy to understand and operate for employees with minimal technical knowledge.	A Streamlit-based interface provides simple forms, upload functions, and dashboards that allow users to interact with the system easily.
Scalability	The system should support increased data volume as more employees and environmental data are added over time.	The system can be deployed in cloud environments and can migrate to larger databases if organizational data grows significantly.

3.3. Out of Scope / Future Enhancements

- The system supports direct integration with IoT devices for automatic environmental data collection
- The system provides predictive analytics for forecasting future carbon emissions or tax liabilities
- The system can be done as a mobile application
- The waste scanning feature estimates weight and classify waste types
- The system includes advanced visualisation dashboards for long-term sustainability tracking

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

- 4.1.1. **Grayscale Rollout & A/B Testing:** In this project, new updates such as improvements to AI analysis, waste scanning, or report generation can first be tested with a limited number of employees or departments. This allows the development team to monitor system stability, detect errors, and collect user feedback before the feature is widely implemented. By analyzing user responses and system performance, the team can determine which version

provides better accuracy, usability, or efficiency before making it the default implementation.

- 4.1.2. **Emergency Rollback (ER) & Golden Release:** If a new AI model update produces incorrect carbon emission calculations, the development team can immediately roll back to the previous working version to maintain system reliability. The Golden Release ensures that core features such as data collection, emission calculations, and carbon tax report generation operate correctly before the system is widely adopted within the organization.

4.1.3. **Priority Matrix:**

Priority	Category	Why
Tier 1	Strategic Decisions	High cost + carbon + ESG
Tier 2	Operational Optimization	Ongoing improvements
Tier 3	Data & AI Tools	Supporting layer

4.2. **Monitoring:**

Monitoring tools can track system performance, API response times, and database activity. Logging mechanisms should also be implemented to record user actions, document uploads, and AI analysis requests.

By maintaining detailed system logs, administrators can identify potential issues such as failed document processing or AI service interruptions. Monitoring also supports auditing and regulatory compliance by maintaining records of environmental data processing activities.

4.3. **Assumptions:**

Several assumptions were made during the design of the system. First, it is assumed that organizations will provide accurate operational data related to procurement and environmental activities. Without accurate data, the AI analysis may produce misleading results. Another assumption is that the external AI service will remain accessible and stable throughout system operation. Since the AI model is accessed through an external API, system functionality depends on the availability of this service. It is also assumed that environmental regulations will remain relatively stable over time. If regulations change significantly, the knowledge base database must be updated with new policy documents.

- 4.4. **External Dependencies:** Do list all the external tools that are applied to your system to function.

Tools	Purpose	Risks
-------	---------	-------

Streamlit	Frontend UI for chat, dashboard, carbon calculator	Limited scalability, not ideal for production multi-user systems, session-state issues
SQLite	Local database for materials, LCA data, submissions, knowledge base	Not scalable, concurrency issues, risk of data corruption under load
pdfplumber	Extract text + tables from policy PDFs into knowledge base	Poor handling of complex/scanned PDFs, inconsistent extraction accuracy
Ilmu AI API	Main LLM (GLM-5.1) for chat analysis, procurement report, reasoning	API downtime, cost scaling, vendor lock-in, response inconsistency
Groq API	Vision AI for receipt extraction + waste identification	Accuracy risk (hallucination), latency, dependency on external uptime
requests	Sends API calls to Ilmu AI & Groq	No retry logic → fragile network handling
FPDF	Generates carbon tax reports (PDF output)	Limited formatting, encoding issues, not enterprise-grade reporting
ZhipuAI SDK	Client wrapper for ILMU/GLM API (Data_extract.py)	Version mismatch risk, dependency break if API changes
Base64 encoding	Converts images for API (vision input)	Large payload size → slower performance
JSON	Data exchange between AI, storage, and frontend	Parsing errors if AI returns invalid JSON
Regex (re module)	Extract structured data from AI responses	Fragile to format changes, breaks easily
OS module	File handling, directory management	Path issues across environments

Datetime module	Timestamping logs and reports	Minor risk (timezone inconsistency)
------------------------	-------------------------------	-------------------------------------

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:

Day 1, Day 2: Project Planning

Day 3 - Day 7: Project Flow Planning (Unable to proceed due to late API distribution)

Day 8, Day 9: Project Development

Day 10: Finalising and Documentation

5.2. Team Members Role:

Chow Hong Yan: Task Distribution; Project Developer

Yip Gui Rong: Project Developer; Flow Designer

Sit Jia En: Flow Designer; Research; Pitch Deck

Tan Hing Huang: Research; Documentation